第 13 章

客户端检测

本章内容

- 使用能力检测
- 用户代理检测的历史
- 软件与硬件检测
- 检测策略

虽然浏览器厂商齐心协力想要实现一致的接口，但事实上仍然是每家浏览器都有自己的长处与不足。跨平台的浏览器尽管版本相同，但总会存在不同的问题。这些差异迫使 Web 开发者要么面向最大公约数而设计，要么（更常见地）使用各种方法来检测客户端，以克服或避免这些缺陷。

客户端检测一直是 Web 开发中饱受争议的话题，这些话题普遍围绕所有浏览器应支持一系列公共特性，理想情况下是这样的。而现实当中，浏览器之间的差异和莫名其妙的行为，让客户端检测变成一种补救措施，而且也成为了开发策略的重要一环。如今，浏览器之间的差异相对 IE 大溃败以前已经好很多了，但浏览器间的不一致性依旧是 Web 开发中的常见主题。

要检测当前的浏览器有很多方法，每一种都有各自的长处和不足。问题的关键在于知道客户端检测应该是解决问题的最后一个举措。任何时候，只要有更普适的方案可选，都应该毫不犹豫地选择。首先要设计最常用的方案，然后再考虑为特定的浏览器进行补救。

13.1 能力检测

能力检测（又称特性检测）即在 JavaScript 运行时中使用一套简单的检测逻辑，测试浏览器是否支持某种特性。这种方式不要求事先知道特定浏览器的信息，只需检测自己关心的能力是否存在即可。能力检测的基本模式如下：

```
if (object.propertyInQuestion) {
  // 使用 object.propertyInQuestion
}
```

比如，IE5 之前的版本中没有 `document.getElementById()` 这个 DOM 方法，但可以通过 `document.all` 属性实现同样的功能。为此，可以进行如下能力检测：

```
function getElement(id) {
  if (document.getElementById) {
    return document.getElementById(id);
  } else if (document.all) {
    return document.all[id];
  } else {
    throw new Error("No way to retrieve element!");
  }
}
```

这个 getElement()函数的目的是根据给定的 ID 获取元素。因为标准的方式是使用 document.getElementById()，所以首先测试它。如果这个函数存在（不是 undefined），那就使用这个方法；否则检测 document.all 是否存在，如果存在则使用。如果这两个能力都不存在（基本上不可能），则抛出错误说明功能无法实现。

能力检测的关键是理解两个重要概念。首先，如前所述，应该先检测最常用的方式。在前面的例子中就是先检测 document.getElementById()再检测 document.all。测试最常用的方案可以优化代码执行，这是因为在多数情况下都可以避免无谓检测。

其次是必须检测切实需要的特性。某个能力存在并不代表别的能力也存在。比如下面的例子：

```
function getWindowWidth() {
  if (document.all) { // 假设 IE
    return document.documentElement.clientWidth; // 不正确的用法!
  } else {
    return window.innerWidth;
  }
}
```

这个例子展示了不正确的能力检测方式。getWindowWidth()函数首先检测 document.all 是否存在，如果存在则返回 document.documentElement.clientWidth，理由是 IE8 及更低版本不支持 window.innerWidth。这个例子的问题在于检测到 document.all 存在并不意味着浏览器是 IE。事实，也可能是某个早期版本的 Opera，既支持 document.all 也支持 windown.innerWidth。

13.1.1 安全能力检测

能力检测最有效的场景是检测能力是否存在的同时，验证其是否能够展现出预期的行为。前一节中的例子依赖将测试对象的成员转换类型，然后再确定它是否存在。虽然这样能够确定检测的对象成员存在，但不能确定它就是你想要的。来看下面的例子，这个函数尝试检测某个对象是否可以排序：

```
// 不要这样做! 错误的能力检测，只能检测到能力是否存在
function isSortable(object) {
  return !!object.sort;
}
```

这个函数尝试通过检测对象上是否有 sort()方法来确定它是否支持排序。问题在于，即使这个对象有一个 sort 属性，这个函数也会返回 true：

```
let result = isSortable({ sort: true });
```

简单地测试到一个属性存在并不代表这个对象就可以排序。更好的方式是检测 sort 是不是函数：

```
// 好一些，检测 sort 是不是函数
function isSortable(object) {
  return typeof object.sort == "function";
}
```

上面的代码中使用的 typeof 操作符可以确定 sort 是不是函数，从而确认是否可以调用它对数据进行排序。

进行能力检测时应该尽量使用 typeof 操作符，但光有它还不够。尤其是某些宿主对象并不保证对 typeof 测试返回合理的值。最有名的例子就是 Internet Explorer（IE）。在多数浏览器中，下面的代码都会在 document.createElement()存在时返回 true：

```
// 不适用于 IE8 及更低版本
function hasCreateElement() {
  return typeof document.createElement == "function";
}
```

但在 IE8 及更低版本中，这个函数会返回 `false`。这是因为 `typeof document.createElement` 返回`"object"`而非`"function"`。前面提到过，DOM 对象是宿主对象，而宿主对象在 IE8 及更低版本中是通过 COM 而非 JScript 实现的。因此，`document.createElement()`函数被实现为 COM 对象，`typeof` 返回`"object"`。IE9 对 DOM 方法会返回`"function"`。

> **注意**　要深入了解 JavaScript 能力检测，推荐阅读 Peter Michaux 的文章“Feature Detection—State of the Art Browser Scripting”。

13.1.2　基于能力检测进行浏览器分析

虽然可能有人觉得能力检测类似于黑科技，但恰当地使用能力检测可以精准地分析运行代码的浏览器。使用能力检测而非用户代理检测的优点在于，伪造用户代理字符串很简单，而伪造能够欺骗能力检测的浏览器特性却很难。

1. 检测特性

可以按照能力将浏览器归类。如果你的应用程序需要使用特定的浏览器能力，那么最好集中检测所有能力，而不是等到用的时候再重复检测。比如：

```
// 检测浏览器是否支持 Netscape 式的插件
let hasNSPlugins = !!(navigator.plugins && navigator.plugins.length);

// 检测浏览器是否具有 DOM Level 1 能力
let hasDOM1 = !!(document.getElementById && document.createElement &&
         document.getElementsByTagName);
```

这个例子完成了两项检测：一项是确定浏览器是否支持 Netscape 式的插件，另一项是检测浏览器是否具有 DOM Level 1 能力。保存在变量中的布尔值可以用在后面的条件语句中，这样比重复检测省事多了。

2. 检测浏览器

可以根据对浏览器特性的检测并与已知特性对比，确认用户使用的是什么浏览器。这样可以获得比用户代码嗅探（稍后讨论）更准确的结果。但未来的浏览器版本可能不适用于这套方案。

下面来看一个例子，根据不同浏览器独有的行为推断出浏览器的身份。这里故意没有使用 `navigator.userAgent` 属性，后面会讨论它：

```
class BrowserDetector {
  constructor() {
    // 测试条件编译
    // IE6~10 支持
    this.isIE_Gte6Lte10 = /*@cc_on!@*/false;

    // 测试 documentMode
    // IE7~11 支持
    this.isIE_Gte7Lte11 = !!document.documentMode;
```

图灵社区会员 aSINKz(1561821892@qq.com) 专享 尊重版权

```
    // 测试 StyleMedia 构造函数
    // Edge 20 及以上版本支持
    this.isEdge_Gte20 = !!window.StyleMedia;

    // 测试 Firefox 专有扩展安装 API
    // 所有版本的 Firefox 都支持
    this.isFirefox_Gte1 = typeof InstallTrigger !== 'undefined';

    // 测试 chrome 对象及其 webstore 属性
    // Opera 的某些版本有 window.chrome，但没有 window.chrome.webstore
    // 所有版本的 Chrome 都支持
    this.isChrome_Gte1 = !!window.chrome && !!window.chrome.webstore;

    // Safari 早期版本会给构造函数的标签符追加"Constructor"字样，如：
    // window.Element.toString(); // [object ElementConstructor]
    // Safari 3~9.1 支持
    this.isSafari_Gte3Lte9_1 = /constructor/i.test(window.Element);

    // 推送通知 API 暴露在 window 对象上
    // 使用默认参数值以避免对 undefined 调用 toString()
    // Safari 7.1 及以上版本支持
    this.isSafari_Gte7_1 =
        (({pushNotification = {}} = {}) =>
          pushNotification.toString() == '[object SafariRemoteNotification]'
        )(window.safari);

    // 测试 addons 属性
    // Opera 20 及以上版本支持
    this.isOpera_Gte20 = !!window.opr && !!window.opr.addons;
  }

  isIE() { return this.isIE_Gte6Lte10 || this.isIE_Gte7Lte11; }
  isEdge() { return this.isEdge_Gte20 && !this.isIE(); }
  isFirefox() { return this.isFirefox_Gte1; }
  isChrome() { return this.isChrome_Gte1; }
  isSafari() { return this.isSafari_Gte3Lte9_1 || this.isSafari_Gte7_1; }
  isOpera() { return this.isOpera_Gte20; }
}
```

这个类暴露的通用浏览器检测方法使用了检测浏览器范围的能力测试。随着浏览器的变迁及发展，可以不断调整底层检测逻辑，但主要的 API 可以保持不变。

3. 能力检测的局限

通过检测一种或一组能力，并不总能确定使用的是哪种浏览器。以下“浏览器检测”代码（或其他类似代码）经常出现在很多网站中，但都没有正确使用能力检测：

```
// 不要这样做！不够特殊
let isFirefox = !!(navigator.vendor && navigator.vendorSub);

// 不要这样做！假设太多
let isIE = !!(document.all && document.uniqueID);
```

这是错误使用能力检测的典型示例。过去，Firefox 可以通过 `navigator.vendor` 和 `navigator.vendorSub` 来检测，但后来 `Safari` 也实现了同样的属性，于是这段代码就会产生误报。为确定 IE，这段代码检测了 `document.all` 和 `document.uniqueID`。这是假设 IE 将来的版本中还会继续存在这两个属性，而且其他浏览器也不会实现它们。不过这两个检测都使用双重否定操作符来产生布尔值（这

样可以生成便于存储和访问的结果）。

> **注意**　能力检测最适合用于决定下一步该怎么做，而不一定能够作为辨识浏览器的标志。

13.2　用户代理检测

用户代理检测通过浏览器的用户代理字符串确定使用的是什么浏览器。用户代理字符串包含在每个 HTTP 请求的头部，在 JavaScript 中可以通过 `navigator.userAgent` 访问。在服务器端，常见的做法是根据接收到的用户代理字符串确定浏览器并执行相应操作。而在客户端，用户代理检测被认为是不可靠的，只应该在没有其他选项时再考虑。

用户代理字符串最受争议的地方就是，在很长一段时间里，浏览器都通过在用户代理字符串包含错误或误导性信息来欺骗服务器。要理解背后的原因，必须回顾一下自 Web 出现之后用户代理字符串的历史。

13.2.1　用户代理的历史

HTTP 规范（1.0 和 1.1）要求浏览器应该向服务器发送包含浏览器名称和版本信息的简短字符串。RFC 2616（HTTP 1.1）是这样描述用户代理字符串的：

> 产品标记用于通过软件名称和版本来标识通信产品的身份。多数使用产品标记的字段也允许列出属于应用主要部分的子产品，以空格分隔。按照约定，产品按照标识应用重要程度的先后顺序列出。

这个规范进一步要求用户代理字符串应该是“标记/版本”形式的产品列表。但现实当中的用户代理字符串远没有那么简单。

1. 早期浏览器

美国国家超级计算应用中心（NCSA，National Center for Supercomputing Applications）发布于 1993 年的 Mosaic 是早期 Web 浏览器的代表，其用户代理字符串相当简单，类似于：

```
Mosaic/0.9
```

虽然在不同操作系统和平台中可能会有所不同，但基本形式都是这么简单直接。斜杠前是产品名称（有时候可能是“NCSA Mosaic”之类的），斜杠后是产品版本。

在网景公司准备开发浏览器时，代号确定为“Mozilla”（Mosaic Killer 的简写）。第一个公开发行版 Netscape Navigator 2 的用户代理字符串是这样的：

```
Mozilla/Version [Language] (Platform; Encryption)
```

网景公司遵守了将产品名称和版本作为用户代理字符串的规定，但又在后面添加了如下信息。

- Language：语言代码，表示浏览器的目标使用语言。
- Platform：表示浏览器所在的操作系统和/或平台。
- Encryption：包含的安全加密类型，可能的值是 U（128 位加密）、I（40 位加密）和 N（无加密）。

Netscape Navigator 2 的典型用户代理字符串如下所示：

```
Mozilla/2.02 [fr] (WinNT; I)
```

这个字符串表示 Netscape Navigator 2.02，在主要使用法语地区的发行，运行在 Windows NT 上，40 位加密。总体上看，通过产品名称还是很容易知道这是什么浏览器的。

2. Netscape Navigator 3 和 IE3

1996 年，Netscape Navigator 3 发布之后超过 Mosaic 成为最受欢迎的浏览器。其用户代理字符串也发生了一些小变化，删除了语言信息，并将操作系统或系统 CPU 信息（OS-or-CPU description）等列为可选信息。此时的格式如下：

```
Mozilla/Version (Platform; Encryption [; OS-or-CPU description])
```

运行在 Windows 系统上的 Netscape Navigator 3 的典型用户代理字符串如下：

```
Mozilla/3.0 (Win95; U)
```

这个字符串表示 Netscape Navigator 3 运行在 Windows 95 上，采用了 128 位加密。注意在 Windows 系统上，没有“OS-or-CPU”部分。

Netscape Navigator 3 发布后不久，微软也首次对外发布了 IE3。这是因为当时 Netscape Navigator 是市场占有率最高的浏览器，很多服务器在返回网页之前都会特意检测其用户代理字符串。如果 IE 因此打不开网页，那么这个当时初出茅庐的浏览器就会遭受重创。为此，IE 就在用户代理字符串中添加了兼容 Netscape 用户代理字符串的内容。结果格式为：

```
Mozilla/2.0 (compatible; MSIE Version; Operating System)
```

比如，Windows 95 平台上的 IE3.02 的用户代理字符串如下：

```
Mozilla/2.0 (compatible; MSIE 3.02; Windows 95)
```

当时的大多数浏览器检测程序都只看用户代理字符串中的产品名称，因此 IE 成功地将自己伪装成了 Mozilla，也就是 Netscape Navigator。这个做法引发了一些争议，因为它违反了浏览器标识的初衷。另外，真正的浏览器版本也跑到了字符串中间。

这个字符串中还有一个地方很有意思，即它将自己标识为 Mozilla 2.0 而不是 3.0。3.0 是当时市面上使用最多的版本，按理说使用这个版本更合逻辑。背后的原因至今也没有揭开，不过很可能就是当事人一时大意造成的。

3. Netscape Communicator 4 和 IE4~8

1997 年 8 月，Netscape Communicator 4 发布（这次发布将 Navigator 改成了 Communicator）。Netscape 在这个版本中仍然沿用了上一个版本的格式：

```
Mozilla/Version (Platform; Encryption [; OS-or-CPU description])
```

比如，Windows 98 上的第 4 版，其用户代理字符串就是这样的：

```
Mozilla/4.0 (Win98; I)
```

如果发布了补丁，则相应增加版本号，比如下面是 4.79 版的字符串：

```
Mozilla/4.79 (Win98; I)
```

微软在发布 IE4 时只更新了版本，格式不变：

```
Mozilla/4.0 (compatible; MSIE Version; Operating System)
```

比如，Windows 98 上运行的 IE4 的字符串如下：

```
Mozilla/4.0 (compatible; MSIE 4.0; Windows 98)
```

更新版本号之后，IE 的版本号跟 Mozilla 的就一致了，识别同为第 4 代的两款浏览器也方便了。可是，这种版本同步就此打住。在 IE4.5（只针对 Mac）面世时，Mozilla 的版本号还是 4，IE 的版本号却变了：

```
Mozilla/4.0 (compatible; MSIE 4.5; Mac_PowerPC)
```

直到 IE7，Mozilla 的版本号就没有变过，比如：

```
Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 5.1)
```

IE8 在用户代理字符串中添加了额外的标识“Trident”，就是浏览器渲染引擎的代号。格式变成：

```
Mozilla/4.0 (compatible; MSIE Version; Operating System; Trident/TridentVersion)
```

比如：

```
Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 5.1; Trident/4.0)
```

这个新增的“Trident”是为了让开发者知道什么时候 IE8 运行兼容模式。在兼容模式下，MSIE 的版本会变成 7，但 Trident 的版本不变：

```
Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 5.1; Trident/4.0)
```

添加这个标识之后，就可以确定浏览器究竟是 IE7（没有“Trident”），还是 IE8 运行在兼容模式。

IE9 稍微升级了一下用户代理字符串的格式。Mozilla 的版本增加到了 5.0，Trident 的版本号也增加到了 5.0。IE9 的默认用户代理字符串是这样的：

```
Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1; Trident/5.0)
```

如果 IE9 运行兼容模式，则会恢复旧版的 Mozilla 和 MSIE 版本号，但 Trident 的版本号还是 5.0。比如，下面就是 IE9 运行在 IE7 兼容模式下的字符串：

```
Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 6.1; Trident/5.0)
```

所有这些改变都是为了让之前的用户代理检测脚本正常运作，同时还能为新脚本提供额外的信息。

4. Gecko

Gecko 渲染引擎是 Firefox 的核心。Gecko 最初是作为通用 Mozilla 浏览器（即后来的 Netscape 6）的一部分开发的。有一个针对 Netscape 6 的用户代理字符串规范，规定了未来的版本应该如何构造这个字符串。新的格式与之前一直沿用到 4.*x* 版的格式有了很大出入：

```
Mozilla/MozillaVersion (Platform; Encryption; OS-or-CPU; Language;
  PrereleaseVersion)Gecko/GeckoVersion
  ApplicationProduct/ApplicationProductVersion
```

这个复杂的用户代理字符串包含了不少想法。下表列出了其中每一部分的含义。

字　符　串	是否必需	说　　明
MozillaVersion	是	Mozilla 版本
Platform	是	浏览器所在的平台。可能的值包括 Windows、Mac 和 X11（UNIX X-Windows）
Encryption	是	加密能力：U 表示 128 位，I 表示 40 位，N 表示无加密
OS-or-CPU	是	浏览器所在的操作系统或计算机处理器类型。如果是 Windows 平台，则这里是 Windows 的版本（如 WinNT、Win95）。如果是 Mac 平台，则这里是 CPU 类型（如 68k、PPC for PowerPC 或 MacIntel）。如果是 X11 平台，则这里是通过 uname -sm 命名得到的 UNIX 操作系统名

（续）

字 符 串	是否必需	说 明
Language	是	浏览器的目标使用语言
Prerelease Version	否	最初的设想是 Mozilla 预发布版的版本号，现在表示 Gecko 引擎的版本号
GeckoVersion	是	以 yyyymmdd 格式的日期表示的 Gecko 渲染引擎的版本
ApplicationProduct	否	使用 Gecko 的产品名称。可能是 Netscape、Firefox 等
ApplicationProductVersion	否	ApplicationProduct 的版本，区别于 MozillaVersion 和 GeckoVersion

要更好地理解 Gecko 的用户代理字符串，最好是看几个不同的基于 Gecko 的浏览器返回的字符串。

Windowx XP 上的 Netscape 6.21：

```
Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:0.9.4) Gecko/20011128
  Netscape6/6.2.1
```

Linux 上的 SeaMonkey 1.1a：

```
Mozilla/5.0 (X11; U; Linux i686; en-US; rv:1.8.1b2) Gecko/20060823 SeaMonkey/1.1a
```

Windows XP 上的 Firefox 2.0.0.11：

```
Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.8.1.11) Gecko/20071127
  Firefox/2.0.0.11
```

Mac OS X 上的 Camino 1.5.1：

```
Mozilla/5.0 (Macintosh; U; Intel Mac OS X; en; rv:1.8.1.6) Gecko/20070809
  Camino/1.5.1
```

所有这些字符串都表示使用的是基于 Gecko 的浏览器（只是版本不同）。有时候，相比于知道特定的浏览器，知道是不是基于 Gecko 才更重要。从第一个基于 Gecko 的浏览器发布开始，Mozilla 版本就是 5.0，一直没有变过。以后也不太可能会变。

在 Firefox 4 发布时，Mozilla 简化了用户代理字符串。主要变化包括以下几方面。

- 去掉了语言标记（即前面例子中的`"en-US"`）。
- 在浏览器使用强加密时去掉加密标记（因为是默认了）。这意味着 `I` 和 `N` 还可能出现，但 `U` 不可能出现了。
- 去掉了 Windows 平台上的平台标记，这是因为跟 OS-or-CPU 部分重复了，否则两个地方都会有 `Windows`。
- GeckoVersion 固定为`"Gecko/20100101"`。

下面是 Firefox 4 中用户代理字符串的例子：

```
Mozilla/5.0 (Windows NT 6.1; rv:2.0.1) Gecko/20100101 Firefox 4.0.1
```

5. WebKit

2003 年，苹果宣布将发布自己的浏览器 Safari。Safari 的渲染引擎叫 WebKit，是基于 Linux 平台浏览器 Konqueror 使用的渲染引擎 KHTML 开发的。几年后，WebKit 又拆分出自己的开源项目，专注于渲染引擎开发。

这个新浏览器和渲染引擎的开发者也面临与当初 IE3.0 时代同样的问题：怎样才能保证浏览器不被排除在流行的站点之外。答案就是在用户代理字符串中添加足够多的信息，让网站知道这个浏览器与其他浏览器是兼容的。于是 Safari 就有了下面这样的用户代理字符串：

```
Mozilla/5.0 (Platform; Encryption; OS-or-CPU; Language)
  AppleWebKit/AppleWebKitVersion (KHTML, like Gecko) Safari/SafariVersion
```

下面是一个实际的例子：

```
Mozilla/5.0 (Macintosh; U; PPC Mac OS X; en) AppleWebKit/124 (KHTML, like Gecko)
  Safari/125.1
```

这个字符串也很长，不仅包括苹果 WebKit 的版本，也包含 Safari 的版本。一开始还有是否需要将浏览器标识为 Mozilla 的争论，但考虑到兼容性很快就达成了一致。现在，所有基于 WebKit 的浏览器都将自己标识为 Mozilla 5.0，与所有基于 Gecko 的浏览器一样。Safari 版本通常是浏览器的构建编号，不一定表示发布的版本号。比如 Safari 1.25 在用户代理字符串中的版本是 125.1，但也不一定始终这样对应。

Safari 用户代理字符串中最受争议的部分是在 1.0 预发布版中添加的`"(KHTML, like Gecko)"`。由于有意想让客户端和服务器把 Safari 当成基于 Gecko 的浏览器（好像光添加`"Mozilla/5.0"`还不够），苹果也招来了很多开发者的反对。苹果的回应与微软当初 IE 遭受质疑时一样：Safari 与 Mozilla 兼容，不能让网站以为用户使用了不受支持的浏览器而把 Safari 排斥在外。

Safari 的用户代理字符串在第 3 版时有所改进。下面的版本标记现在用来表示 Safari 实际的版本号：

```
Mozilla/5.0 (Macintosh; U; PPC Mac OS X; en) AppleWebKit/522.15.5
  (KHTML, like Gecko) Version/3.0.3 Safari/522.15.5
```

注意这个变化只针对 Safari 而不包括 WebKit。因此，其他基于 WebKit 的浏览器可能不会有这个变化。一般来说，与 Gecko 一样，通常识别是不是 WebKit 比识别是不是 Safari 更重要。

6. Konqueror

Konqueror 是与 KDE Linux 桌面环境打包发布的浏览器，基于开源渲染引擎 KHTML。虽然只有 Linux 平台的版本，Konqueror 的用户却不少。为实现最大化兼容，Konqueror 决定采用 Internet Explore 的用户代理字符串格式：

```
Mozilla/5.0 (compatible; Konqueror/Version; OS-or-CPU)
```

不过，Konqueror 3.2 为了与 WebKit 就标识为 KHTML 保持一致，也对格式做了一点修改：

```
Mozilla/5.0 (compatible; Konqueror/Version; OS-or-CPU) KHTML/KHTMLVersion
  (like Gecko)
```

下面是一个例子：

```
Mozilla/5.0 (compatible; Konqueror/3.5; SunOS) KHTML/3.5.0 (like Gecko)
```

Konqueror 和 KHTML 的版本号通常是一致的，有时候也只有子版本号不同。比如 Konqueror 是 3.5，而 KHTML 是 3.5.1。

7. Chrome

谷歌的 Chrome 浏览器使用 Blink 作为渲染引擎，使用 V8 作为 JavaScript 引擎。Chrome 的用户代理字符串包含所有 WebKit 的信息，另外又加上了 Chrome 及其版本的信息。其格式如下所示：

```
Mozilla/5.0 (Platform; Encryption; OS-or-CPU; Language)
  AppleWebKit/AppleWebKitVersion (KHTML, like Gecko)
  Chrome/ChromeVersion Safari/SafariVersion
```

以下是 Chrome 7 完整的用户代理字符串：

```
Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US) AppleWebKit/534.7
  (KHTML, like Gecko) Chrome/7.0.517.44 Safari/534.7
```

其中的 Safari 版本和 WebKit 版本有可能始终保持一致，但也不能肯定。

8. Opera

在用户代理字符串方面引发争议最大的一个浏览器就是 Opera。Opera 默认的用户代理字符串是所有现代浏览器中最符合逻辑的，因为它正确标识了自己和版本。在 Opera 8 之前，其用户代理字符串都是这个格式：

```
Opera/Version (OS-or-CPU; Encryption) [Language]
```

比如，Windows XP 上的 Opera 7.54 的字符串是这样的：

```
Opera/7.54 (Windows NT 5.1; U) [en]
```

Opera 8 发布后，语言标记从括号外挪到了括号内，目的是与其他浏览器保持一致：

```
Opera/Version (OS-or-CPU; Encryption; Language)
```

Windows XP 上的 Opera 8 的字符串是这样的：

```
Opera/8.0 (Windows NT 5.1; U; en)
```

默认情况下，Opera 会返回这个简单的用户代理字符串。这是唯一一个使用产品名称和版本完全标识自身的主流浏览器。不过，与其他浏览器一样，Opera 也遇到了使用这种字符串的问题。虽然从技术角度看这是正确的，但网上已经有了很多浏览器检测代码只考虑 Mozilla 这个产品名称。还有不少代码专门针对 IE 或 Gecko。为了不让这些检测代码判断错误，Opera 坚持使用唯一标识自身的字符串。

从 Opera 9 开始，Opera 也采用了两个策略改变自己的字符串。一是把自己标识为别的浏览器，如 Firefox 或 IE。这时候的字符串跟 Firefox 和 IE 的一样，只不过末尾会多一个"Opera"及其版本号。比如：

```
Mozilla/5.0 (Windows NT 5.1; U; en; rv:1.8.1) Gecko/20061208 Firefox/2.0.0
  Opera 9.50

Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1; en) Opera 9.50
```

第一个字符串把 Opera 9.5 标识为 Firefox 2，同时保持了 Opera 版本信息。第二个字符串把 Opera 9.5 标识为 IE6，也保持了 Opera 版本信息。虽然这些字符串可以通过针对 Firefox 和 IE 的测试，但也可以被识别为 Opera。

另一个策略是伪装成 Firefox 或 IE。这种情况下的用户代理字符串与 Firefox 和 IE 返回的一样，末尾也没有"Opera"及其版本信息。这样就根本没办法区分 Opera 与其他浏览器了。更严重的是，Opera 还会根据访问的网站不同设置不同的用户代理字符串，却不通知用户。比如，导航到 My Yahoo 网站会导致 Opera 将自己伪装成 Firefox。这就导致很难通过用户代理字符串来识别 Opera。

> **注意** 在 Opera 7 之前的版本中，Opera 可以解析 Windows 操作系统字符串的含义。比如，Windows NT 5.1 实际上表示 Windows XP。因此 Opera 6 的用户代理字符串中会包含 Windows XP 而不是 Windows NT 5.1。为了与其他浏览器表现更一致，Opera 7 及后来的版本就改为使用官方报告的操作系统字符串，而不是自己转换的了。

Opera 10 又修改了字符串格式，变成了下面这样：

```
Opera/9.80 (OS-or-CPU; Encryption; Language) Presto/PrestoVersion Version/Version
```

注意开头的版本号 Opera/9.80 是固定不变的。Opera 没有 9.8 这个版本，但 Opera 工程师担心某些浏览器检测脚本会错误地把 Opera/10.0 当成 Opera 1 而不是 Opera 10。因此，Opera 10 新增了额外的

Presto 标识（Presto 是 Opera 的渲染引擎）和版本标识。比如，下面是 Windows 7 上的 Opera 10.63 的字符串：

```
Opera/9.80 (Windows NT 6.1; U; en) Presto/2.6.30 Version/10.63
```

Opera 最近的版本已经改为在更标准的字符串末尾追加"OPR"标识符和版本号。这样，除了末尾的"OPR"标识符和版本号，字符串的其他部分与 WebKit 浏览器是类似的。下面就是 Windows 10 上的 Opera 52 的用户代理字符串：

```
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/65.0.3325.181 Safari/537.36 OPR/52.0.2871.64
```

9. iOS 与 Android

iOS 和 Android 移动操作系统上默认的浏览器都是基于 WebKit 的，因此具有与相应桌面浏览器一样的用户代理字符串。iOS 设备遵循以下基本格式：

```
Mozilla/5.0 (Platform; Encryption; OS-or-CPU like Mac OS X; Language)
  AppleWebKit/AppleWebKitVersion (KHTML, like Gecko) Version/BrowserVersion
  Mobile/MobileVersion Safari/SafariVersion
```

注意其中用于辅助判断 Mac 操作系统的"like Mac OS X"和"Mobile"相关的标识。这里的 Mobile 标识除了说明这是移动 WebKit 之外并没有什么用。平台可能是"iPhone"、"iPod"或"iPad"，因设备而异。例如：

```
Mozilla/5.0 (iPhone; U; CPU iPhone OS 3_0 like Mac OS X; en-us)
  AppleWebKit/528.18 (KHTML, like Gecko) Version/4.0 Mobile/7A341 Safari/528.16
```

注意在 iOS 3 以前，操作系统的版本号不会出现在用户代理字符串中。

默认的 Android 浏览器通常与 iOS 上的浏览器格式相同，只是没有 Mobile 后面的版本号（"Mobile"标识还有）。例如：

```
Mozilla/5.0 (Linux; U; Android 2.2; en-us; Nexus One Build/FRF91)
  AppleWebKit/533.1 (KHTML, like Gecko) Version/4.0 Mobile Safari/533.1
```

这个用户代理字符串是谷歌 Nexus One 手机上的默认浏览器的。不过，其他 Android 设备上的浏览器也遵循相同的模式。

13.2.2　浏览器分析

想要知道自己代码运行在什么浏览器上，大部分开发者会分析 window.navigator.userAgent 返回的字符串值。所有浏览器都会提供这个值，如果相信这些返回值并基于给定的一组浏览器检测这个字符串，最终会得到关于浏览器和操作系统的比较精确的结果。

相比于能力检测，用户代理检测还是有一定优势的。能力检测可以保证脚本不必理会浏览器而正常执行。现代浏览器用户代理字符串的过去、现在和未来格式都是有章可循的，我们能够利用它们准确识别浏览器。

1. 伪造用户代理

通过检测用户代理来识别浏览器并不是完美的方式，毕竟这个字符串是可以造假的。只不过实现 window.navigator 对象的浏览器（即所有现代浏览器）都会提供 userAgent 这个只读属性。因此，简单地给这个属性设置其他值不会有效：

```
console.log(window.navigator.userAgent);
// Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/65.0.3325.181 Safari/537.36

window.navigator.userAgent = 'foobar';

console.log(window.navigator.userAgent);
// Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/65.0.3325.181 Safari/537.36
```

不过，通过简单的办法可以绕过这个限制。比如，有些浏览器提供伪私有的`__defineGetter__`方法，利用它可以篡改用户代理字符串：

```
console.log(window.navigator.userAgent);
// Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/65.0.3325.181 Safari/537.36

window.navigator.__defineGetter__('userAgent', () => 'foobar');

console.log(window.navigator.userAgent);
// foobar
```

对付这种造假是一件吃力不讨好的事。检测用户代理是否以这种方式被篡改过是可能的，但总体来看还是一场猫捉老鼠的游戏。

与其劳心费力检测造假，不如更好地专注于浏览器识别。如果相信浏览器返回的用户代理字符串，那就可以用它来判断浏览器。如果怀疑脚本或浏览器可能篡改这个值，那最好还是使用能力检测。

2. 分析浏览器

通过解析浏览器返回的用户代理字符串，可以极其准确地推断出下列相关的环境信息：

- 浏览器
- 浏览器版本
- 浏览器渲染引擎
- 设备类型（桌面/移动）
- 设备生产商
- 设备型号
- 操作系统
- 操作系统版本

当然，新浏览器、新操作系统和新硬件设备随时可能出现，其中很多可能有着类似但并不相同的用户代理字符串。因此，用户代理解析程序需要与时俱进，频繁更新，以免落伍。自己手写的解析程序如果不及时更新或修订，很容易就过时了。本书上一版写过一个用户代理解析程序，但这一版并不推荐读者自己从头再写一个。相反，这里推荐一些 GitHub 上维护比较频繁的第三方用户代理解析程序：

- Bowser
- UAParser.js
- Platform.js
- CURRENT-DEVICE
- Google Closure
- Mootools

> **注意**　Mozilla 维基有一个页面“Compatibility/UADetectionLibraries”，其中提供了用户代理解析程序的列表，可以用来识别 Mozilla 浏览器（甚至所有主流浏览器）。这些解析程序是按照语言分组的。这个页面好像维护不频繁，但其中给出了所有主流的解析库。（注意 JavaScript 部分包含客户端库和 Node.js 库。）GitHub 上的文章“Are We Detectable Yet?”中还有一张可视化的表格，能让我们对这些库的检测能力一目了然。

13.3　软件与硬件检测

现代浏览器提供了一组与页面执行环境相关的信息，包括浏览器、操作系统、硬件和周边设备信息。这些属性可以通过暴露在 `window.navigator` 上的一组 API 获得。不过，这些 API 的跨浏览器支持还不够好，远未达到标准化的程度。

> **注意**　强烈建议在使用这些 API 之前先检测它们是否存在，因为其中多数都不是强制性的，且很多浏览器没有支持。另外，本节介绍的特性有时候不一定可靠。

13.3.1　识别浏览器与操作系统

特性检测和用户代理字符串解析是当前常用的两种识别浏览器的方式。而 `navigator` 和 `screen` 对象也提供了关于页面所在软件环境的信息。

1. navigator.oscpu

`navigator.oscpu` 属性是一个字符串，通常对应用户代理字符串中操作系统/系统架构相关信息。根据 HTML 实时标准：

> `oscpu` 属性的获取方法必须返回空字符串或者表示浏览器所在平台的字符串，比如`"Windows NT 10.0; Win64; x64"`或`"Linux x86_64"`。

比如，Windows 10 上的 Firefox 的 `oscpu` 属性应该对应于以下加粗的部分：

```
console.log(navigator.userAgent);
"Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:58.0) Gecko/20100101 Firefox/58.0"
console.log(navigator.oscpu);
"Windows NT 10.0; Win64; x64"
```

2. navigator.vendor

`navigator.vendor` 属性是一个字符串，通常包含浏览器开发商信息。返回这个字符串是浏览器 `navigator` 兼容模式的一个功能。根据 HTML 实时标准：

> `navigator.vendor` 返回一个空字符串，也可能返回字符串`"Apple Computer, Inc."`或字符串`"Google Inc."`。

例如，Chrome 中的这个 `navigator.vendor` 属性返回下面的字符串：

```
console.log(navigator.vendor); // "Google Inc."
```

3. navigator.platform

`navigator.platform` 属性是一个字符串，通常表示浏览器所在的操作系统。根据 HTML 实时标准：

> navigator.platform 必须返回一个字符串或表示浏览器所在平台的字符串，例如"MacIntel"、"Win32"、"FreeBSD i386"或"WebTV OS"。

例如，Windows 系统下 Chrome 中的这个 navigator.platform 属性返回下面的字符串：

```
console.log(navigator.platform); // "Win32"
```

4. screen.colorDepth 和 screen.pixelDepth

screen.colorDepth 和 screen.pixelDepth 返回一样的值，即显示器每像素颜色的位深。根据 CSS 对象模型（CSSOM）规范：

> screen.colorDepth 和 screen.pixelDepth 属性应该返回输出设备中每像素用于显示颜色的位数，不包含 alpha 通道。

Chrome 中这两个属性的值如下所示：

```
console.log(screen.colorDepth); // 24
console.log(screen.pixelDepth); // 24
```

5. screen.orientation

screen.orientation 属性返回一个 ScreenOrientation 对象，其中包含 Screen Orientation API 定义的屏幕信息。这里面最有意思的属性是 angle 和 type，前者返回相对于默认状态下屏幕的角度，后者返回以下 4 种枚举值之一：

- portrait-primary
- portrait-secondary
- landscape-primary
- landscape-secondary

例如，在 Chrome 移动版中，screen.orientation 返回的信息如下：

```
// 垂直看
console.log(screen.orientation.type);   // portrait-primary
console.log(screen.orientation.angle);  // 0

// 向左转
console.log(screen.orientation.type);   // landscape-primary
console.log(screen.orientation.angle);  // 90

// 向右转
console.log(screen.orientation.type);   // landscape-secondary
console.log(screen.orientation.angle);  // 270
```

根据规范，这些值的初始化取决于浏览器和设备状态。因此，不能假设 portrait-primary 和 0 始终是初始值。这两个值主要用于确定设备旋转后浏览器的朝向变化。

13.3.2 浏览器元数据

navigator 对象暴露出一些 API，可以提供浏览器和操作系统的状态信息。

1. Geolocation API

navigator.geolocation 属性暴露了 Geolocation API，可以让浏览器脚本感知当前设备的地理位置。这个 API 只在安全执行环境（通过 HTTPS 获取的脚本）中可用。

这个 API 可以查询宿主系统并尽可能精确地返回设备的位置信息。根据宿主系统的硬件和配置，返

回结果的精度可能不一样。手机 GPS 的坐标系统可能具有极高的精度，而 IP 地址的精度就要差很多。根据 Geolocation API 规范：

> 地理位置信息的主要来源是 GPS 和 IP 地址、射频识别（RFID）、Wi-Fi 及蓝牙 Mac 地址、GSM/CDMA 蜂窝 ID 以及用户输入等信息。

> **注意**　浏览器也可能会利用 Google Location Service（Chrome 和 Firefox）等服务确定位置。有时候，你可能会发现自己并没有 GPS，但浏览器给出的坐标却非常精确。浏览器会收集所有可用的无线网络，包括 Wi-Fi 和蜂窝信号。拿到这些信息后，再去查询网络数据库。这样就可以精确地报告出你的设备位置。

要获取浏览器当前的位置，可以使用 `getCurrentPosition()` 方法。这个方法返回一个 `Coordinates` 对象，其中包含的信息不一定完全依赖宿主系统的能力：

```
// getCurrentPosition()会以position对象为参数调用传入的回调函数
navigator.geolocation.getCurrentPosition((position) => p = position);
```

这个 `position` 对象中有一个表示查询时间的时间戳，以及包含坐标信息的 `Coordinates` 对象：

```
console.log(p.timestamp);  // 1525364883361
console.log(p.coords);     // Coordinates {...}
```

`Coordinates` 对象中包含标准格式的经度和纬度，以及以米为单位的精度。精度同样以确定设备位置的机制来判定。

```
console.log(p.coords.latitude, p.coords.longitude);   // 37.4854409, -122.2325506
console.log(p.coords.accuracy);                       // 58
```

`Coordinates` 对象包含一个 `altitude`（海拔高度）属性，是相对于 1984 世界大地坐标系（World Geodetic System，1984）地球表面的以米为单位的距离。此外也有一个 `altitudeAccuracy` 属性，这个精度值单位也是米。为了取得 `Coordinates` 中包含的这些信息，当前设备必须具备相应的能力（比如 GPS 或高度计）。很多设备因为没有能力测量高度，所以这两个值经常有一个或两个是空的。

```
console.log(p.coords.altitude);          // -8.800000190734863
console.log(p.coords.altitudeAccuracy);  // 200
```

`Coordinates` 对象包含一个 `speed` 属性，表示设备每秒移动的速度。还有一个 `heading`（朝向）属性，表示相对于正北方向移动的角度（0 ≤ `heading` < 360）。为获取这些信息，当前设备必须具备相应的能力（比如加速计或指南针）。很多设备因为没有能力测量高度，所以这两个值经常有一个是空的，或者两个都是空的。

> **注意**　设备不会根据两点的向量来测量速度和朝向。不过，如果可能的话，可以尝试基于两次连续的测量数据得到的向量来手动计算。当然，如果向量的精度不够，那么计算结果的精度肯定也不够。

获取浏览器地理位置并不能保证成功。因此 `getCurrentPosition()` 方法也接收失败回调函数作为第二个参数，这个函数会收到一个 `PositionError` 对象。在失败的情况下，`PositionError` 对象中会包含一个 `code` 属性和一个 `message` 属性，后者包含对错误的简短描述。`code` 属性是一个整数，表示以下 3 种错误。

图灵社区会员 aSINKz(1561821892@qq.com) 专享 尊重版权

- PERMISSION_DENIED：浏览器未被允许访问设备位置。页面第一次尝试访问 Geolocation API 时，浏览器会弹出确认对话框取得用户授权（每个域分别获取）。如果返回了这个错误码，则要么是用户不同意授权，要么是在不安全的环境下访问了 Geolocation API。message 属性还会提供额外信息。
- POSITION_UNAVAILABLE：系统无法返回任何位置信息。这个错误码可能代表各种失败原因，但相对来说并不常见，因为只要设备能上网，就至少可以根据 IP 地址返回一个低精度的坐标。
- TIMEOUT：系统不能在超时时间内返回位置信息。关于如何配置超时，会在后面介绍。

```
// 浏览器会弹出确认对话框请用户允许访问 Geolocation API
// 这个例子显示了用户拒绝之后的结果
navigator.geolocation.getCurrentPosition(
  () => {},
  (e) => {
    console.log(e.code);      // 1
    console.log(e.message);   // User denied Geolocation
  }
);

// 这个例子展示了在不安全的上下文中执行代码的结果
navigator.geolocation.getCurrentPosition(
  () => {},
  (e) => {
    console.log(e.code);      // 1
    console.log(e.message);   // Only secure origins are allowed
  }
);
```

Geolocation API 位置请求可以使用 PositionOptions 对象来配置，作为第三个参数提供。这个对象支持以下 3 个属性。

- enableHighAccuracy：布尔值，true 表示返回的值应该尽量精确，默认值为 false。默认情况下，设备通常会选择最快、最省电的方式返回坐标。这通常意味着返回的是不够精确的坐标。比如，在移动设备上，默认位置查询通常只会采用 Wi-Fi 和蜂窝网络的定位信息。而在 enableHighAccuracy 为 true 的情况下，则会使用设备的 GPS 确定设备位置，并返回这些值的混合结果。使用 GPS 会更耗时、耗电，因此在使用 enableHighAccuracy 配置时要仔细权衡一下。
- timeout：毫秒，表示在以 TIMEOUT 状态调用错误回调函数之前等待的最长时间。默认值是 0xFFFFFFFF（$2^{32}-1$）。0 表示完全跳过系统调用而立即以 TIMEOUT 调用错误回调函数。
- maximumAge：毫秒，表示返回坐标的最长有效期，默认值为 0。因为查询设备位置会消耗资源，所以系统通常会缓存坐标并在下次返回缓存的值（遵从位置缓存失效策略）。系统会计算缓存期，如果 Geolocation API 请求的配置要求比缓存的结果更新，则系统会重新查询并返回值。0 表示强制系统忽略缓存的值，每次都重新查询。而 Infinity 会阻止系统重新查询，只会返回缓存的值。JavaScript 可以通过检查 Position 对象的 timestamp 属性值是否重复来判断返回的是不是缓存值。

2. Connection State 和 NetworkInformation API

浏览器会跟踪网络连接状态并以两种方式暴露这些信息：连接事件和 navigator.onLine 属性。在设备连接到网络时，浏览器会记录这个事实并在 window 对象上触发 online 事件。相应地，当设备

图灵社区会员 aSINKz(1561821892@qq.com) 专享 尊重版权

断开网络连接后，浏览器会在 window 对象上触发 offline 事件。任何时候，都可以通过 navigator.onLine 属性来确定浏览器的联网状态。这个属性返回一个布尔值，表示浏览器是否联网。

```
const connectionStateChange = () => console.log(navigator.onLine);

window.addEventListener('online', connectionStateChange);
window.addEventListener('offline', connectionStateChange);

// 设备联网时：
// true

// 设备断网时：
// false
```

当然，到底怎么才算联网取决于浏览器与系统实现。有些浏览器可能会认为只要连接到局域网就算“在线”，而不管是否真正接入了互联网。

navigator 对象还暴露了 NetworkInformation API，可以通过 navigator.connection 属性使用。这个 API 提供了一些只读属性，并为连接属性变化事件处理程序定义了一个事件对象。

以下是 NetworkInformation API 暴露的属性。

- downlink：整数，表示当前设备的带宽（以 Mbit/s 为单位），舍入到最接近的 25kbit/s。这个值可能会根据历史网络吞吐量计算，也可能根据连接技术的能力来计算。
- downlinkMax：整数，表示当前设备最大的下行带宽（以 Mbit/s 为单位），根据网络的第一跳来确定。因为第一跳不一定反映端到端的网络速度，所以这个值只能用作粗略的上限值。
- effectiveType：字符串枚举值，表示连接速度和质量。这些值对应不同的蜂窝数据网络连接技术，但也用于分类无线网络。这个值有以下 4 种可能。
  - slow-2g
    - 往返时间 > 2000ms
    - 下行带宽 < 50kbit/s
  - 2g
    - 2000ms > 往返时间 ≥ 1400ms
    - 70kbit/s > 下行带宽 ≥ 50kbit/s
  - 3g
    - 1400ms > 往返时间 ≥ 270ms
    - 700kbit/s > 下行带宽 ≥ 70kbit/s
  - 4g
    - 270ms > 往返时间 ≥ 0ms
    - 下行带宽 ≥ 700kbit/s
- rtt：毫秒，表示当前网络实际的往返时间，舍入为最接近的 25 毫秒。这个值可能根据历史网络吞吐量计算，也可能根据连接技术的能力来计算。
- type：字符串枚举值，表示网络连接技术。这个值可能为下列值之一。
  - bluetooth：蓝牙。
  - cellular：蜂窝。
  - ethernet：以太网。
  - none：无网络连接。相当于 navigator.onLine === false。

- mixed：多种网络混合。
- other：其他。
- unknown：不确定。
- wifi：Wi-Fi。
- wimax：WiMAX。

- saveData：布尔值，表示用户设备是否启用了“节流”（reduced data）模式。
- onchange：事件处理程序，会在任何连接状态变化时激发一个 change 事件。可以通过 navigator.connection.addEventListener('change',changeHandler)或 navigator.connection.onchange = changeHandler 等方式使用。

3. Battery Status API

浏览器可以访问设备电池及充电状态的信息。navigator.getBattery()方法会返回一个期约实例，解决为一个 BatteryManager 对象。

```
navigator.getBattery().then((b) => console.log(b));
// BatteryManager { ... }
```

BatteryManager 包含 4 个只读属性，提供了设备电池的相关信息。

- charging：布尔值，表示设备当前是否正接入电源充电。如果设备没有电池，则返回 true。
- chargingTime：整数，表示预计离电池充满还有多少秒。如果电池已充满或设备没有电池，则返回 0。
- dischargingTime：整数，表示预计离电量耗尽还有多少秒。如果设备没有电池，则返回 Infinity。
- level：浮点数，表示电量百分比。电量完全耗尽返回 0.0，电池充满返回 1.0。如果设备没有电池，则返回 1.0。

这个 API 还提供了 4 个事件属性，可用于设置在相应的电池事件发生时调用的回调函数。可以通过给 BatteryManager 添加事件监听器，也可以通过给事件属性赋值来使用这些属性。

- onchargingchange
- onchargingtimechange
- ondischargingtimechange
- onlevelchange

```
navigator.getBattery().then((battery) => {
  // 添加充电状态变化时的处理程序
  const chargingChangeHandler = () => console.log('chargingchange');
  battery.onchargingchange = chargingChangeHandler;
  // 或
  battery.addEventListener('chargingchange', chargingChangeHandler);

  // 添加充电时间变化时的处理程序
  const chargingTimeChangeHandler = () => console.log('chargingtimechange');
  battery.onchargingtimechange = chargingTimeChangeHandler;
  // 或
  battery.addEventListener('chargingtimechange', chargingTimeChangeHandler);

  // 添加放电时间变化时的处理程序
  const dischargingTimeChangeHandler = () => console.log('dischargingtimechange');
  battery.ondischargingtimechange = dischargingTimeChangeHandler;
  // 或
  battery.addEventListener('dischargingtimechange', dischargingTimeChangeHandler);
```

```
  // 添加电量百分比变化时的处理程序
  const levelChangeHandler = () => console.log('levelchange');
  battery.onlevelchange = levelChangeHandler;
  // 或
  battery.addEventListener('levelchange', levelChangeHandler);
});
```

13.3.3　硬件

浏览器检测硬件的能力相当有限。不过，navigator 对象还是通过一些属性提供了基本信息。

1. 处理器核心数

navigator.hardwareConcurrency 属性返回浏览器支持的逻辑处理器核心数量，包含表示核心数的一个整数值（如果核心数无法确定，这个值就是 1）。关键在于，这个值表示浏览器可以并行执行的最大工作线程数量，不一定是实际的 CPU 核心数。

2. 设备内存大小

navigator.deviceMemory 属性返回设备大致的系统内存大小，包含单位为 GB 的浮点数（舍入为最接近的 2 的幂：512MB 返回 0.5，4GB 返回 4）。

3. 最大触点数

navigator.maxTouchPoints 属性返回触摸屏支持的最大关联触点数量，包含一个整数值。

13.4　小结

客户端检测是 JavaScript 中争议最多的话题之一。因为不同浏览器之间存在差异，所以经常需要根据浏览器的能力来编写不同的代码。客户端检测有不少方式，但下面两种用得最多。

- **能力检测**，在使用之前先测试浏览器的特定能力。例如，脚本可以在调用某个函数之前先检查它是否存在。这种客户端检测方式可以让开发者不必考虑特定的浏览器或版本，而只需关注某些能力是否存在。能力检测不能精确地反映特定的浏览器或版本。
- **用户代理检测**，通过用户代理字符串确定浏览器。用户代理字符串包含关于浏览器的很多信息，通常包括浏览器、平台、操作系统和浏览器版本。用户代理字符串有一个相当长的发展史，很多浏览器都试图欺骗网站相信自己是别的浏览器。用户代理检测也比较麻烦，特别是涉及 Opera 会在代理字符串中隐藏自己信息的时候。即使如此，用户代理字符串也可以用来确定浏览器使用的渲染引擎以及平台，包括移动设备和游戏机。

在选择客户端检测方法时，首选是使用能力检测。特殊能力检测要放在次要位置，作为决定代码逻辑的参考。用户代理检测是最后一个选择，因为它过于依赖用户代理字符串。

浏览器也提供了一些软件和硬件相关的信息。这些信息通过 screen 和 navigator 对象暴露出来。利用这些 API，可以获取关于操作系统、浏览器、硬件、设备位置、电池状态等方面的准确信息。